



Universidad
FUCS

[VIGILADA MINEDUCACIÓN]

ProfundaMente

Taller: IA Aplicada a la Investigación en Salud

John Jaime Sprockel, Md ESP, MSc, MBA. Director Laboratorio ProfundaMente

Sesión 4

Análisis de bases de datos apoyado en chatbot

Python reproducible en Google Colab para investigación en salud

Laboratorio guiado | 2 horas | Presentación inicial: 5 minutos

Taller de aplicaciones prácticas de IA en investigación en salud



FUCS

Propósito y resultados de aprendizaje

Un laboratorio para construir un análisis trazable, verificable y reutilizable.

1

Crear un cuaderno en Google Colab y vincularlo con Drive

2

Cargar, explorar y preparar una base clínica

3

Describir población, desenlace y valores faltantes

4

Entrenar modelos predictivos iniciales

5

Interpretar resultados con criterio metodológico

Resultado integrador

Cada estudiante finalizará con un cuaderno funcional que documenta el flujo completo: carga de datos, exploración, limpieza, análisis descriptivo, preprocesamiento, modelamiento e interpretación crítica.

Programación paso a paso en Python, con celdas explicativas en Markdown.
Uso del chatbot como asistente para explicar, corregir, documentar y revisar código.
Énfasis en transparencia, reproducibilidad y prevención de errores metodológicos.

Caso de trabajo: base clínica COVID-19

Datos demográficos, clínicos, factores de riesgo, examen físico, laboratorios y desenlace.

711

registros de pacientes

47

variables analíticas

COVID

desenlace binario

Punto de partida para el análisis

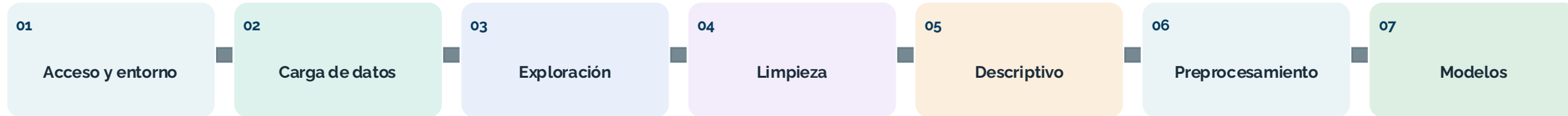
Variable de desenlace: presencia o ausencia de COVID-19.
La base permite practicar exploración, limpieza, descripción y predicción.
El desenlace está desbalanceado: conviene usar partición estratificada y métricas complementarias.

Pregunta guía: ¿qué variables ayudan a predecir COVID-19 en esta base?



Flujo de trabajo de ciencia de datos

La sesión sigue una secuencia reproducible, desde el archivo hasta la interpretación.



Principio conductor

Cada paso debe quedar documentado con código ejecutable y una explicación breve en Markdown. Esto permite que otro investigador pueda revisar, repetir o adaptar el análisis.

Rol del chatbot en cada fase

- Generar código inicial.
- Explicar errores y proponer correcciones.
- Redactar notas metodológicas.
- Sugerir visualizaciones y métricas.
- Revisar riesgos de sesgo y sobreajuste.

Presentación inicial: 5 minutos

Mensaje para orientar el laboratorio antes de empezar a programar.

1. Qué haremos hoy

Base de datos → análisis reproducible.
Exploración, limpieza y descripción.
Tres modelos predictivos iniciales.

2. Cómo usaremos IA

Como tutor de código.
Como revisor metodológico.
Como apoyo para documentar.

Advertencia crítica

El chatbot puede acelerar el trabajo, pero no garantiza exactitud. El estudiante debe verificar código, supuestos, métricas e interpretación.

Producto final

Cuaderno Colab funcional.
Tablas y gráficos iniciales.
Modelos entrenados.
Métricas comparativas.
Comentario crítico final.

Acceso a Google Colab y vinculación con Drive

Preparar el entorno antes de analizar la base.

```
from google.colab import drive
drive.mount('/content/drive')
```



Prompt de apoyo

“Actúa como tutor de Python para investigación en salud. Necesito cargar una base desde Google Drive en Colab. Explícame qué hace el código y cómo verificar que el archivo está disponible.”

Producto: cuaderno creado, Drive montado y archivo localizado.

Pasos operativos

Ingresa a Google Colab con cuenta institucional o personal.
Crear un cuaderno nuevo y nombrarlo con el formato del taller.
Crear carpeta en Drive y cargar allí la base de datos.
Montar Drive y verificar la ruta del archivo.

Carga de datos y exploración inicial

Convertir el archivo en un DataFrame y diagnosticar su estructura.

```
import pandas as pd
import numpy as np

ruta = '/content/drive/MyDrive/Taller_IA_Salud/para IA.xlsx'
df = pd.read_excel(ruta)
df.head()
```

```
df.shape
df.columns
df.info()
df['COVID'].value_counts(normalize=True) * 100
```

Qué debe verificar el estudiante

Número de filas y columnas.
Nombres y tipos de variables.
Primeras filas para detectar errores de lectura.
Distribución del desenlace COVID.
Valores perdidos y variables con codificación dudosa.

Idea pedagógica: antes de modelar, primero se comprende la base.

Limpieza y preparación básica

Estandarizar nombres, revisar duplicados y clasificar variables.

Código nuclear

```
df.columns = (
    df.columns
    .str.strip()
    .str.replace(' ', '_')
    .str.replace('-', '_')
    .str.replace('/', '_')
)

df.duplicated().sum()
```

```
variables_numericas = df.select_dtypes(
    include=['int64', 'float64']
).columns.tolist()

variables_categoricas = df.select_dtypes(
    include=['object', 'category', 'bool']
).columns.tolist()
```

Lista de chequeo antes de modelar

Confirmar que COVID es el desenlace y no está duplicado.

Detectar variables que puedan generar fuga de información.

Diferenciar numéricas, categóricas y binarias.

Revisar duplicados, datos imposibles y valores extremos.

Definir qué variables se incluirán como predictores.

Riesgo clave

Incluir una variable creada después del diagnóstico puede inflar artificialmente el desempeño del modelo.

Análisis descriptivo general y por desenlace

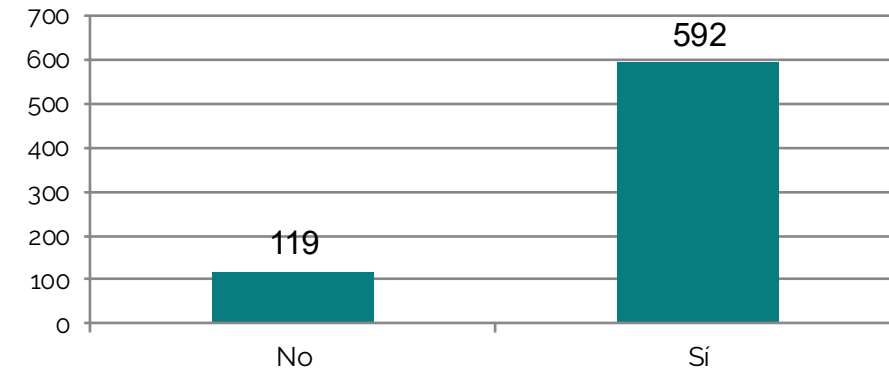
Antes de predecir, describir la población y explorar diferencias.

```
df.describe().T  
  
df.groupby('COVID').describe().T  
  
for col in variables_categoricas:  
    print(col)  
    print(df[col].value_counts(dropna=False))
```

Interpretación esperada

Describir sin inferir causalidad. Diferenciar hallazgos exploratorios de asociaciones y de desempeño predictivo.

Distribución del desenlace



Usar tablas resumidas y gráficos simples.
Documentar qué variables tienen más datos faltantes.
Evitar sobreinterpretar comparaciones no ajustadas.

Preprocesamiento reproducible

Separar predictores y desenlace, dividir datos y construir pipelines.

```
y = df['COVID']
X = df.drop(columns=['COVID'])

from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

```
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
```

Decisiones metodológicas

Partición entrenamiento/prueba con estratificación por desenlace.

Imputación de faltantes solo dentro del pipeline.

Escalamiento de variables numéricas para regresión logística.

Codificación one-hot para variables categóricas.

El preprocesamiento se aprende en entrenamiento y se aplica a prueba.

El pipeline reduce errores y mejora la reproducibilidad del análisis.

Modelos predictivos iniciales

Tres enfoques complementarios para un desenlace binario.

Modelo	Propósito pedagógico	Punto de atención
Regresión logística multivariada	Modelo interpretable de referencia	Revisar colinealidad, codificación y pesos de clase
XGBoost	Modelo flexible basado en árboles potenciados	Puede sobreajustar si no se valida adecuadamente
LightGBM	Modelo eficiente para datos tabulares	Requiere evaluación cuidadosa en clases desbalanceadas

Código conceptual

```
modelo = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('classifier', Clasificador(...))
])

modelo.fit(X_train, y_train)
```

Advertencia docente

Comparar modelos no significa seleccionar automáticamente el de mayor métrica. La selección debe considerar interpretabilidad, estabilidad, validación y pertinencia clínica.

Evaluación de desempeño

Usar métricas adecuadas cuando el desenlace está desbalanceado.

Métrica	Qué responde
Accuracy	Proporción total de clasificaciones correctas; puede ser engañosa si hay desbalance.
Balanced accuracy	Promedio entre sensibilidad y especificidad.
Sensibilidad / recall	Capacidad para identificar verdaderos positivos.
Especificidad	Capacidad para identificar verdaderos negativos.
F1	Balance entre precisión y sensibilidad.
ROC AUC	Discriminación global a través de umbrales.

Función común de evaluación

```
def evaluar_modelo(nombre, modelo, X_test, y_test):
    y_pred = modelo.predict(X_test)
    y_proba = modelo.predict_proba(X_test)[:, 1]
    return {
        'modelo': nombre,
        'balanced_accuracy': balanced_accuracy_score(y_test,
y_pred),
        'sensibilidad': recall_score(y_test, y_pred),
        'f1': f1_score(y_test, y_pred),
        'roc_auc': roc_auc_score(y_test, y_proba)
    }
```

No concluir utilidad clínica sin validación externa y evaluación del impacto.

Cómo dialogar con el chatbot durante el análisis

Prompts transversales para aprender, depurar y documentar.

Explicar código

"Explicame línea por línea este código y sus supuestos."

Corregir error

"Te comparto el código y el mensaje de error. Propón una corrección."

Documentar celda

"Redacta una celda Markdown para explicar este paso del análisis."

Interpretar tabla

"Interpreta estos resultados sin afirmar causalidad."

Revisar riesgos

"Identifica fuga de información, sobreajuste y sesgos potenciales."

Regla de uso: pedir explicación y verificación, no solo copiar código.

Cierre: producto mínimo y continuidad

La sesión termina con un cuaderno reproducible y una interpretación crítica.

Cuaderno esperado

Introducción: base, procedencia y desenlace.
Carga de datos y exploración inicial.
Valores faltantes y descriptivo por desenlace.
Preprocesamiento y partición entrenamiento/prueba.
Tres modelos: regresión logística, XGBoost y LightGBM.
Tabla comparativa de métricas e interpretación crítica.

Criterios de calidad

Código ejecutado sin errores.
Celdas Markdown antes de cada bloque.
Resultados no sobreinterpretados.
Reconocimiento explícito de limitaciones.
Prompts usados durante el análisis.

Mensaje final: la IA acelera el proceso, pero la reproducibilidad y la validez dependen del investigador.

Tarea posterior: entregar el cuaderno Colab con análisis ejecutado, tabla de modelos, interpretación de máximo 300 palabras y reflexión sobre el uso del chatbot.